

`</>` beginner's handbook

JavaScript from Zero

A hands-on, example-driven guide to learning JavaScript from scratch — variables to your first script in the browser.

មគ្គុទ្ទេសក៍រៀនភាសា JavaScript ចាប់ពីដំបូង

ES2020+

no experience needed

runs in any browser

What's Inside

1	What is JavaScript & how to run it (ភាសា JavaScript)	§1
2	Variables (អថេរ) — let, const, var	§2
3	Data types (ប្រភេទទិន្នន័យ)	§3
4	Operators & conditionals (លក្ខខណ្ឌ)	§4
5	Loops (រង្វិលជុំ)	§5
6	Functions (អនុគមន៍)	§6
7	Arrays (គ្រាប់) & array methods	§7
8	Objects (វត្ថុ)	§8
9	The DOM & events (ព្រឹត្តិការណ៍)	§9
10	Browser storage (ការផ្ទុកទិន្នន័យ)	§10
11	Two tiny real projects (គម្រោង)	§11
12	Where to go next	§12

What Is JavaScript? (តើ JavaScript គឺជាអ្វី?)

The language that powers interactive websites.

ភាសាសរសេរកម្មវិធីដែលធ្វើឱ្យគេហទំព័រមានអន្តរកម្ម។

JavaScript is the programming language that makes websites interactive — every button click, dropdown, form validation, and live-updating page uses it. It runs in every web browser and, thanks to Node.js, on servers too.

The fastest way to try it: the browser console (ក្នុងស្វ័យកម្មវិធីរុករក)

Open any web page, right-click → **Inspect** → click the **Console** tab. Type code and hit Enter — it runs instantly. No installation needed.

```
console
console.log("Hello, JavaScript!");
console.warn("This is a warning");
console.error("This is an error");
```

OUTPUT

```
Hello, JavaScript!
△ This is a warning
✖ This is an error
```

Running a script in an HTML page

```
index.html
<!DOCTYPE html>
<html>
  <body>
    <h1>My Page</h1>
    <script src="script.js"></script>
  </body>
</html>
```

Comments (សេចក្តីអធិប្បាយ) — notes the computer ignores

```
comments.js

// a single-line comment

/* a comment
   that spans
   multiple lines */

const price = 10; // inline comment explaining the value
```

Tip — Save that as `index.html`, create a `script.js` next to it, and double-click the HTML file to open it in your browser. That's your whole setup.

Variables (អថេរ)

Containers that store data you can use and change.

ធុងផ្ទុកទិន្នន័យ ដែលអាចប្រើប្រាស់ និងផ្លាស់ប្តូរបាន។

Variables (អថេរ) store data so you can reuse and change it. JavaScript has three ways to declare one:

```
variables.js
let age = 27;           // can change later
const name = "Tim";     // cannot be reassigned
var oldStyle = "avoid"; // legacy, avoid in new code

age = 28;               // works, let is assignable
// name = "Sam";        error - const can't be reassigned
```

Keyword	Reassignable?	When to use
<code>let</code> (អាចប្តូរ)	Yes	Default choice for values that change
<code>const</code> (មិនប្តូរ)	No	Default choice — use unless you need reassignment
<code>var</code>	Yes	Legacy only — skip it in new code

More examples — combining variables

```
combine.js

const firstName = "Tim";
const city = "Phnom Penh";

// template literal — the modern way to build strings
const intro = `${firstName} lives in ${city}.`;
console.log(intro);

// swapping two variables using destructuring
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b); // 2 1
```

OUTPUT

```
Tim lives in Phnom Penh.
2 1
```

Rule of thumb — reach for `const` by default, switch to `let` only when the value truly needs to change.

Data Types (ប្រភេទទិន្នន័យ)

The different kinds of values JavaScript works with.

ប្រភេទតម្លៃផ្សេងៗគ្នា ដែល JavaScript អាចដំណើរការបាន។

```
types.js

const aString = "hello";      // text
const aNumber = 42;           // integers & decimals, one type
const aBoolean = true;        // true / false
const aList = [1, 2, 3];      // array
const aObject = { city: "Phnom Penh" }; // object
const nothing = null;          // intentionally empty
let notSet;                     // undefined until assigned

console.log(typeof aNumber); // "number"
```

JavaScript is **dynamically typed** (ប្រភេទថាមវន្ត) — you don't declare a type up front, and a variable can hold any type. Use `typeof` to check what something currently is.

More examples — type coercion pitfalls

```
coercion.js

"5" + 1;      // "51" - string + number = string
"5" - 1;      // 4    - minus forces a number
1 + 1;        // 2    - plain math

Array.isArray([1, 2]); // true
typeof [1, 2];         // "object" - arrays are objects too!
typeof null;           // "object" - a famous JS quirk
```

Watch out — `+` with a string always joins text together. Use `Number(value)` to convert text into a real number first if you want math.

Operators & Conditionals (ប្រមាណវិធី និងលក្ខខណ្ឌ)

Compare values and make decisions in your code.

ប្រៀបធៀបតម្លៃ និងធ្វើការសម្រេចចិត្តនៅក្នុងកូដ។

```
operators.js

10 + 5;    10 - 5;    10 * 5;    10 / 5;    10 % 3; // remainder → 1

5 === 5;   // true  - strict equals (use this one)
5 == "5";  // true  - loose equals, avoid it
5 !== 6;   // true  - not equal
```

if / else (ប្រសិនបើ / បើមិនដូច្នោះ)

```
conditionals.js

const hour = 14;

if (hour < 12) {
  console.log("Good morning");
} else if (hour < 18) {
  console.log("Good afternoon");
} else {
  console.log("Good evening");
}
```

OUTPUT

Good afternoon

More examples — ternary & switch

ternary-switch.js

```
// ternary – a compact if/else in one line
const age = 20;
const label = age >= 18 ? "adult" : "minor";

// switch – clean way to handle many exact cases
const day = "Mon";
switch (day) {
  case "Sat":
  case "Sun":
    console.log("Weekend");
    break;
  default:
    console.log("Weekday");
}
```

OUTPUT

Weekday

Loops (រង្វិលជុំ)

Repeat an action without rewriting the same code.

ធ្វើសកម្មភាពម្តងទៀត ដោយមិនចាំបាច់សរសេរកូដដដែលឡើងវិញ។

```
loops.js

// for – when you know how many times
for (let i = 1; i <= 3; i++) {
  console.log("Lap " + i);
}

// while – when you don't know how many times
let stock = 3;
while (stock > 0) {
  stock--;
}

// for...of – the easiest way to loop over an array
for (const fruit of ["apple", "pear"]) {
  console.log(fruit);
}
```

OUTPUT

```
Lap 1
Lap 2
Lap 3
apple
pear
```

More examples — break & continue

```
break-continue.js

for (let i = 1; i <= 5; i++) {
  if (i === 3) continue; // skip 3
  if (i === 5) break;    // stop before 5
  console.log(i);
}
```

OUTPUT

```
1
2
4
```

Functions (អនុគមន៍)

Reusable blocks of code you can call anytime.

ប្លុកកូដដែលអាចប្រើប្រាស់ឡើងវិញបានគ្រប់ពេល។

A function (អនុគមន៍) is a reusable block of code.

Functions are commonly grouped into 4 types (៤ប្រភេទ)

Type	Example	Short description
Declaration ការប្រកាស	<code>function greet(){} </code>	Named, usable before it's defined (ប្រើបានមុនកន្លែងប្រកាស)
Expression កន្សោម	<code>const greet = function(){} </code>	Stored in a variable, not usable before it (ត្រូវប្រកាសមុនប្រើ)
Arrow ព្រួញ	<code>const greet = () => {} </code>	Short syntax, no own <code>this</code> (សរសេរខ្លី)
Method វិធីសាស្ត្រ	<code>user.greet() </code>	A function attached to an object (ជាប់នឹងវត្ថុ)

```

functions.js

// 1. classic function declaration
function greet(name) {
  return "Hi, " + name;
}

// 2. function expression – stored in a variable
const sayBye = function(name) {
  return "Bye, " + name;
};

// 3. arrow function – shorter, very common today
const add = (a, b) => a + b;

// 4. method – a function that lives inside an object
const bot = {
  reply() { return "beep boop"; }
};

console.log(greet("Tim")); // "Hi, Tim"
console.log(add(2, 3)); // 5
console.log(bot.reply()); // "beep boop"

```

More examples — a function that takes a function

```
callback.js

// a "callback" - passing a function as an argument
function processOrder(item, onDone) {
  console.log("Preparing " + item);
  onDone(item);
}

processOrder("coffee", (item) => {
  console.log(item + " is ready!");
});
```

OUTPUT

```
Preparing coffee
coffee is ready!
```

Tip — Arrow functions (`=>`) are the modern default for short functions. Reach for a classic `function` when you need it hoisted (usable before its definition) or a method on an object.

Arrays (អាន) & Array Methods

Ordered lists of values.

បញ្ជីតម្លៃដែលរៀបតាមលំដាប់។

```
arrays.js

const nums = [4, 8, 15, 16];

nums.push(23);           // add to the end
nums.length;             // 5

const doubled = nums.map(n => n * 2);
const big      = nums.filter(n => n > 10);
const total    = nums.reduce((sum, n) => sum + n, 0);

console.log(doubled); // [8, 16, 30, 32, 46]
console.log(big);     // [15, 16, 23]
console.log(total);   // 66
```

Method	What it does
<code>.map()</code>	Transforms every item, returns a new array
<code>.filter()</code>	Keeps items that pass a test
<code>.reduce()</code>	Folds the array down to a single value
<code>.find()</code>	Returns the first item that matches
<code>.forEach()</code>	Runs code per item, no new array returned

More examples — sort, includes, slice

```
more-arrays.js

const fruits = ["pear", "apple", "mango"];

fruits.includes("mango"); // true
fruits.sort();             // ["apple", "mango", "pear"]
fruits.slice(0, 2);        // ["apple", "mango"] – copy, doesn't change original
fruits.join(", ");        // "apple, mango, pear" – array → string
```

Objects (វត្ថុ)

Grouped data with named properties.

ទិន្នន័យដែលដាក់ជាក្រុម ដោយមានឈ្មោះសម្គាល់។

Objects (វត្ថុ) group related data together using **key: value** pairs.

```
objects.js

const user = {
  name: "Tim",
  role: "developer",
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};

console.log(user.name);    // "Tim"
console.log(user.greet()); // "Hi, I'm Tim"

// destructuring – pull values out directly
const { role } = user;
console.log(role);        // "developer"
```

Note — that backtick string (``Hi, I'm ${this.name}``) is a **template literal** — it lets you drop variables straight into a string with `${ }`.

More examples — keys, values, and spreading

```
object-tools.js

const product = { name: "Mouse", price: 15 };

Object.keys(product);    // ["name", "price"]
Object.values(product);  // ["Mouse", 15]

// spread – copy an object and change one field
const onSale = { ...product, price: 10 };
console.log(onSale);    // { name: "Mouse", price: 10 }
```

The DOM & Events (ព្រឹត្តិការណ៍)

How JavaScript reads and reacts to the page.

របៀបដែល JavaScript អាន និងឆ្លើយតបទៅនឹងទំព័រគេហទំព័រ។

The DOM (Document Object Model) is how JavaScript reads and changes what's on the page.

```
dom.js

const button = document.querySelector("#myBtn");
const title = document.querySelector("h1");

button.addEventListener("click", () => {
  title.textContent = "You clicked it!";
  title.style.color = "#5a5aff";
});
```

`querySelector` finds an element on the page, `addEventListener` reacts to what the user does with it — click, type, hover, submit, and more.

More examples — reading input & creating elements

```
form.js

const input = document.querySelector("#nameInput");
const list = document.querySelector("#list");

document.querySelector("#addBtn").addEventListener("click", () => {
  const li = document.createElement("li");
  li.textContent = input.value; // read what the user typed
  list.appendChild(li); // add it to the page
  input.value = ""; // clear the box
});
```

Browser Storage (ការផ្ទុកទិន្នន័យ)

Save data in the browser between visits.

រក្សាទុកទិន្នន័យនៅក្នុងកម្មវិធីរុករក រវាងការចូលមើលនីមួយៗ។

Besides variables (which disappear on refresh), the browser gives JavaScript a few built-in places to store data.

localStorage (ការផ្ទុករយៈពេលយូរ) — persists forever

```
local-storage.js

localStorage.setItem("username", "Tim");
localStorage.getItem("username"); // "Tim"
localStorage.removeItem("username");
localStorage.clear(); // wipe everything
```

Data saved here stays even after closing the browser or restarting the computer — until code or the user clears it.

sessionStorage (ការផ្ទុករយៈពេលខ្លី) — clears on tab close

```
session-storage.js

sessionStorage.setItem("step", "2");
sessionStorage.getItem("step"); // "2"
// closing this browser tab erases everything automatically
```

Storage	Lifetime	Typical use
localStorage	Until manually cleared	Saved theme, login token, cart
sessionStorage	Until the tab closes	Multi-step form progress
Cookies (ខ្លី)	Set by you, small size	Data the server also needs to read
IndexedDB (មូលដ្ឋានទិន្នន័យ)	Until manually cleared	Larger, structured offline data

Note — both storages only accept strings. To store an object or array, convert it first with `JSON.stringify()`, and read it back with `JSON.parse()`.

Two Tiny Real Projects (គំរោង)

Putting it together — variables, a function, and an event listener:

```
counter.html

<button id="counterBtn">Clicked 0 times</button>

<script>
  let count = 0;
  const btn = document.querySelector("#counterBtn");

  btn.addEventListener("click", () => {
    count++;
    btn.textContent = `Clicked ${count} times`;
  });
</script>
```

Bonus project — a simple to-do list

```
todo.html

<input id="task" placeholder="New task">
<button id="addTask">Add</button>
<ul id="tasks"></ul>

<script>
  const input = document.querySelector("#task");
  const tasks = document.querySelector("#tasks");

  document.querySelector("#addTask").addEventListener("click", () => {
    if (!input.value) return;
    const item = document.createElement("li");
    item.textContent = input.value;
    item.addEventListener("click", () => item.remove()); // click to remove
    tasks.appendChild(item);
    input.value = "";
  });
</script>
```

TRY IT YOURSELF (សាកល្បងដោយខ្លួនឯង)

modify the counter so the button turns red after 5 clicks and resets to 0 after 10.

Hint: you'll need an `if` check inside the click handler.

Where to Go Next

- **Practice daily** — even 20 minutes a day on small exercises beats one long session a week.
- **Build tiny projects** — to-do list, quiz app, calculator. Small and finished beats big and abandoned.
- **Learn Git** — version control early saves pain later.
- **Read real code** — browse small open-source repos on GitHub once the basics feel comfortable.
- **Then explore** — async/await & promises, ES modules, and a framework (React or Vue) once fundamentals are solid.

You now know enough to build things.

Every JavaScript developer started exactly here — variables, functions, and a `console.log`. The fastest way forward from this point is not more reading, it's writing broken code and fixing it.